UAO

UNIVERSIDAD AUTÓNOMA DE OCCIDENTE

FACULTAD DE INGENIERÍA

Programa: Maestría en Inteligencia Artificial y Ciencia de Datos

Asignatura: Gestión de Proyectos en IA y CD

PROPUESTA FORMAL M2: SISTEMA DESACOPLADO DE INSPECCIÓN DE CALIDAD INDUSTRIAL PARA DETECCIÓN DE DEFECTOS SUPERFICIALES EN ACERO LAMINADO CON YOLOV8 / RT-DETR, EXPLICABILIDAD VISUAL Y DESPLIEGUE DOCKER CPU

INTEGRANTES DEL EQUIPO (MÁX. 4):

Omar David Sánchez Jiménez | Código estudiantil: 2260142

Docente: Jan Polanco Velasco

Fecha formal de radicación:
Santiago de Cali, Colombia — 5 de noviembre de 2026

TABLA DE CONTENIDO OBLIGATORIA (6 CAPÍTULOS)

CAPÍTULO 1. CONTEXTO, MODEL CARD Y RESTRICCIONES TÉCNICAS Pág. 1

1.1. Planteamiento del Problema Real, Usuario Afectado y Jerarquía de Modelos (Tiers)
1.2. Ficha del Modelo Base (MaaS)
1.3. Resumen de la Model Card y Análisis Ético / Operativo
1.4. Matriz de Restricciones Técnicas (Software, Hardware y Licenciamiento)

CAPÍTULO 2. OBJETIVOS Y MATRIZ DE ALCANCE Pág. 3

2.1. Formulación de Objetivos (General y Específicos Medibles)
2.2. Matriz de Alcance del Proyecto (Included / Nice to Have / Not Included)

CAPÍTULO 3. CRONOGRAMA A 15 DÍAS Y ANÁLISIS DE RUTA CRÍTICA (CPM) Pág. 5

3.1. Desglose en las Tres Fases Universales del Sprint (15 Días)
3.2. Matriz de Precedencias, Tiempos Tempranos/Tardíos y Holguras (CPM)
3.3. Diagrama de Gantt a 15 Días y Análisis de Tareas Críticas

CAPÍTULO 4. MARCO DE INVESTIGACIÓN Y MAPEO A CRISP-DM Pág. 7

4.1. Evaluación Comparativa de Formatos de Serialización de Modelos
4.2. Dataset Industrial (NEU-DET), Técnicas de Preprocesamiento y Visión por Computador
4.3. Mapeo Sistemático a las 6 Fases de la Metodología CRISP-DM

CAPÍTULO 5. HISTORIAS DE USUARIO, CRITERIOS DE ACEPTACIÓN Y KANBAN Pág. 9

5.1. Estimación Ágil Fibonacci (1:1) y Calculadora de Capacidad del Equipo
5.2. Catálogo Completo de Historias de Usuario, Criterios de Aceptación (AC) y Fibonacci
5.3. Configuración del Tablero Kanban en GitHub Projects y Límites WIP

CAPÍTULO 6. ANEXOS, ENLACES, LICENCIAS Y REFERENCIAS BIBLIOGRÁFICAS Pág. 12

6.1. Enlaces Oficiales al Repositorio, Tablero Kanban, Dataset y Model Card
6.2. Árbol de Directorios Real del Paquete Industrial (src/ Layout) y Evidencia de Tooling
6.3. Matriz de Cumplimiento Normativo (ISO/IEC 42001 y EU AI Act) y Licenciamiento
6.4. Referencias Bibliográficas (Formato IEEE)

Cumplimiento de Especificación Editorial y Técnica (Acto III - Entrega M2 35%):

- Formato: PDF formal, 100% propuesta, sin mapas mentales, extensión <= 15 páginas de contenido.
- Tipografía: Arial | Títulos H1: 14 pt negrita | Subtítulos H2/H3: 12 pt negrita | Cuerpo: 12 pt.
- Espaciado: Interlineado 1.5 líneas (22.5 pt / paso vertical) y márgenes simétricos de 2.5 cm.
- Contenido: Portada (7 ítems), 6 Capítulos, Model Card, Matriz In/Nice/Out, Gantt 15d + CPM, CRISP-DM y Kanban Fibonacci 1:1

CAPÍTULO 1. CONTEXTO, MODEL CARD Y RESTRICCIONES TÉCNICAS

1.1. Planteamiento del Problema Real, Usuario Afectado y Jerarquía de Modelos (Tiers)

En la industria siderúrgica y metalmecánica moderna, la laminación en caliente y en frío de bandas de acero constituye un proceso continuo de alta velocidad (entre 5 y 15 metros por segundo) donde cualquier anomalía superficial compromete la integridad estructural del producto terminado. Los defectos superficiales típicos —tales como agrietamientos térmicos (crazing), inclusiones no metálicas (inclusion), parches de oxidación (patches), picaduras superficiales (pitted surface), cascarilla laminada incrustada (rolled-in scale) y rayaduras mecánicas longitudinales (scratches)— generan pérdidas millonarias por rechazos de lotes completos en sectores automotriz, electrodomésticos y construcción.

Actualmente, los inspectores e ingenieros de aseguramiento de calidad en planta realizan muestreos visuales estroboscópicos o revisiones manuales post-laminación. Este enfoque presenta tres fallos críticos: (1) fatiga visual severa del operario tras turnos superiores a 4 horas, reduciendo la tasa de detección en defectos milimétricos por debajo del 72%; (2) alta variabilidad inter-inspector al clasificar entre cascarilla e inclusión; y (3) latencia de retroalimentación tardía que impide ajustar los rodillos de laminación a tiempo. El usuario directamente afectado es el Ingeniero de Calidad de Planta y el Operario de Línea de Laminación, quienes requieren un sistema automatizado, determinista y auditable que detecte, clasifique y localice espacialmente las 6 tipologías de defectos en menos de 200 milisegundos por captura sin requerir infraestructura GPU costosa en borde.

En consonancia con la Economía de la IA en Tres Niveles (Tiers) expuesta en el Acto I y Acto III de la asignatura, sobrediseñar invocando un modelo fundacional de frontera (Nivel 1: Claude, GPT o Gemini Pro, con latencias de 2 a 15 segundos y alto costo por millón de tokens) o un modelo de conocimiento general (Nivel 2: Gemini Flash o Llama 70B, 500 ms a 1.5 s) para inspeccionar cientos de láminas por minuto destruiría el presupuesto operativo e incumpliría el SLA industrial en tiempo real. Por ello, esta propuesta justifica arquitectónicamente el uso del Nivel 3 (Tareas Especializadas / SLMs y Visión Dedicada): el procesamiento masivo en línea es asumido íntegramente por un detector especializado ligero (YOLOv8n / RT-DETR exportado a ONNX) con latencia inferior a 100 ms y costo marginal cero en CPU local, mientras que la gobernanza de la salida se asegura mediante contratos numéricos deterministas y explicabilidad visual Eigen-CAM / Grad-CAM.

1.2. Ficha del Modelo Base (MaaS)

Siguiendo los criterios de selección de los repositorios Hugging Face Hub y ModelScope del Acto III, se adopta un esquema Model-as-a-Service (MaaS) local basado en pesos preentrenados de arquitectura YOLOv8 de Ultralytics especializados sobre el benchmark siderúrgico NEU Surface Defect Database (NEU-DET), con alternativa interoperable en ModelScope (DAMO-YOLO / RT-DETR).

Atributo Técnico	Especificación Oficial del Modelo Base (MaaS)
Identificación Formal	YOLOv8n-NEU-DET (Surface Defect Detection) / Ultralytics YOLOv8 Nano
Enlaces Oficiales	Hugging Face: https://huggingface.co/ultralyticsplus/yolov8s | ModelScope: https://modelscope.cn/models | GitHub: https://github.com/ultralytics/ultralytics
Autoría e Institución	Ultralytics Inc. & Northeastern University (NEU Song et al., Benchmark Industrial)
Función Matemática	Detección de objetos en una etapa (Anchor-Free Object Detection) y clasificación multiclase sobre K = 6 categorías: regresión de cajas delimitadoras L_CIoU + Distribution Focal Loss (DFL) y Binary Cross-Entropy (BCE) para probabilidades de clase.
Formato de Pesos	Pesos nativos PyTorch (.pt) exportados a formato abierto interoperable ONNX (.onnx) y .safetensors para carga segura sin ejecución arbitraria de pickle.

1.3. Resumen de la Model Card y Análisis Ético / Operativo

Con base en el estándar de documentación de modelos (Mitchell et al., 2019) y las directrices de los marcos ISO/IEC 42001 y EU AI Act, se sintetiza la Model Card del modelo base junto con sus sesgos, limitaciones y métricas de referencia:

Dimensión Model Card	Descripción y Parámetros Verificables
Dataset de Entrenamiento	NEU Surface Defect Database (NEU-DET): 1,800 imágenes en escala de grises (200x200 px nativos), exactamente 300 imágenes por cada una de las 6 clases (Crazing, Inclusion, Patches, Pitted_Surface, Rolled-in_Scale, Scratches) con más de 3,000 cajas delimitadoras anotadas en formato PASCAL VOC / YOLO.
Métricas Reportadas	mAP@50 = 0.785 (78.5%), mAP@50-95 = 0.462, F1-Score macro = 0.768, Latencia de inferencia en CPU (ONNX Runtime x86_64) = 42.5 ms por imagen (512x512).
Sesgos del Dataset	1) Sesgo de Iluminación: Capturas obtenidas bajo iluminación halógena uniforme de laboratorio; variaciones bruscas de reflectancia especular en plantas reales degradan la precisión si no se aplica ecualización adaptativa previa. 2) Sesgo de Aleación: Entrenado en acero laminado en caliente estándar; aceros inoxidables espejo o aluminio anodizado presentan texturas base distintas.
Limitaciones Éticas y Operativas	1) Alta confusión inter-clase entre 'Crazing' y 'Rolled-in_Scale' en resoluciones bajas. 2) Riesgo de sesgo de automatización en operarios que aprueben bobinas defectuosas sin verificar el mapa de saliencia. Se impone supervisión Human-in-the-Loop (HITL) obligatoria.

1.4. Matriz de Restricciones Técnicas (Software, Hardware y Licenciamiento)

Para asegurar la viabilidad operativa en estaciones industriales sin GPU dedicada y el cumplimiento estricto del tooling del Acto II y III, se establecen las siguientes restricciones técnicas:

Categoría	Componente	Restricción y Condición de Operación
Software	Entorno y Gestor	Python >= 3.13 gestionado exclusivamente con uv (uv.lock determinista; prohibido pip install manual o requirements.txt).
Software	Motor e Inferencia	ONNX Runtime CPU / PyTorch / OpenCV Headless >= 4.10.0, FastAPI para microservicio y Streamlit para interfaz visual.
Software	Calidad y Guardia	Estructura obligatoria src/ layout, pytest >= 8.0.0 para pruebas unitarias, Ruff >= 0.9.0 (reglas E, F, I, B) y pre-commit >= 4.0.0.
Hardware Mínimo	Memoria RAM Host	Mínimo 8 GB RAM (16 GB recomendados para ejecución simultánea de contenedor Docker + UI + API).
Hardware Mínimo	Procesador (CPU/VRAM)	CPU x86_64 de 4 núcleos físicos con instrucciones AVX2. Cero requerimiento de VRAM / GPU dedicada (100% compatible con Docker CPU).
Licenciamiento	Código y Modelo	Código propio del repositorio bajo licencia MIT. Modelo YOLOv8 bajo AGPL-3.0 (uso académico/interno abierto; en caso de distribución comercial cerrada se migra el mismo pipeline ONNX a RT-DETR / Apache-2.0 sin alterar el contrato src/). Dataset NEU-DET de acceso público académico.

CAPÍTULO 2. OBJETIVOS Y MATRIZ DE ALCANCE

2.1. Formulación de Objetivos (General y Específicos Medibles)

Objetivo General: Construir, verificar y desplegar un servicio modular desacoplado para la detección y clasificación automática de seis tipos de defectos superficiales en acero laminado (NEU-DET) integrando preprocesamiento adaptativo CLAHE, inferencia en CPU con explicabilidad visual (Eigen-CAM / Grad-CAM), API REST en FastAPI, interfaz operativa en Streamlit y contenedor Docker reproducible.

Objetivo Específico 1 (Pipeline y Contratos): Implementar un pipeline modular de ingesta y preprocesamiento bajo arquitectura src/ layout que estandarice imágenes industriales a tensores float32 de dimensión exacta (512, 512, 1) normalizados en [0.0, 1.0] mediante ecualización adaptativa CLAHE (clipLimit=2.0, tileGridSize=8x8),

validado al 100% con pruebas unitarias automatizadas en pytest.

Objetivo Específico 2 (Inferencia MaaS y Latencia): Desacoplar el motor de inferencia de detección multiclase y generación de mapas de calor de explicabilidad visual garantizando una latencia extremo a extremo inferior a 200 ms por imagen en CPU estándar y un F1-Score / mAP@50 >= 0.75 sobre el conjunto de validación NEU-DET.

Objetivo Específico 3 (Industrialización, Tooling y Despliegue): Empaquetar la solución en un artefacto binario Wheel (.whl) mediante uv build e integrarla en un contenedor Docker CPU no privilegiado gobernado con Ruff, pre-commit y flujo CI/CD en GitHub Actions.

2.2. Matriz de Alcance del Proyecto (Included / Nice to Have / Not Included)

Con el propósito de evitar desviaciones de alcance (Scope Creep) durante el sprint de 15 días calendario y proteger la Ruta Crítica, se formaliza el contrato de alcance en tres niveles:

Categoría de Alcance	Elementos Técnicos y Entregables Comprometidos
Included (Dentro del Alcance Obligatorio)	1) Arquitectura profesional src/sistema_inspeccion_industrial/ con pyproject.toml y uv.lock. 2) Módulo read_img.py para lectura de imágenes industriales en escala de grises y buffers de memoria. 3) Módulo preprocess_img.py con filtro CLAHE, redimensionamiento (512, 512, 1) y validación formal de contrato (dtype float32, rango [0.0, 1.0], finitud sin NaN/Inf). 4) Integración del detector MaaS para las 6 clases de NEU-DET con superposición de mapa térmico de saliencia y cajas delimitadoras. 5) Servicio de inferencia API REST y UI interactiva en Streamlit para operarios de calidad. 6) Suite de pruebas pytest con fixtures sintéticos en conftest.py, configuración de Ruff + pre-commit y Dockerfile CPU funcional.
Nice to Have (Deseable si existe holgura)	1) Cuantización dinámica INT8 del grafo ONNX para reducir latencia por debajo de 25 ms en CPU. 2) Autenticación mediante tokens JWT en los endpoints de inferencia. 3) Panel histórico de telemetría con gráficos de distribución de defectos por lote y latencia p95/p99. 4) Exportación automática de certificados de inspección de calidad en formato PDF por bobina.
Not Included (Excluido Explícitamente)	1) Captura física con cámaras industriales de línea (GigE Vision / GenICam) o control de PLCs en planta. 2) Recolección y etiquetado manual de nuevas muestras siderúrgicas en fábrica. 3) Entrenamiento distribuido desde cero (from scratch) o aprovisionamiento de servidores con GPU dedicada NVIDIA. 4) Despliegue en clústeres Kubernetes multi-nodo en nube pública.

CAPÍTULO 3. CRONOGRAMA A 15 DÍAS Y ANÁLISIS DE RUTA CRÍTICA (CPM)

3.1. Desglose en las Tres Fases Universales del Sprint (15 Días)

El cronograma de ejecución se estructura exactamente en un horizonte de 15 días calendario dividido en las tres fases universales establecidas en el Acto III:

Fase 1 — Preparación (Días 1 a 3): Revisión de literatura y Model Card del detector NEU-DET, descarga y verificación de integridad criptográfica de pesos y muestras crudas del dataset, definición de contratos matemáticos e inicialización del repositorio con uv init --package, pyproject.toml, uv.lock y hooks de pre-commit.

Fase 2 — Desarrollo & Refactoring (Días 4 a 10): Construcción aislada de los módulos del paquete en src/sistema_inspeccion_industrial/ (read_img.py, preprocess_img.py, load_model.py, integrator.py), implementación de contratos numéricos float32, carga segura del modelo MaaS, creación de fixtures en tests/conftest.py y validación continua con uv run pytest y uv run ruff check.

Fase 3 — Implementación & Despliegue (Días 11 a 15): Construcción de la capa de servicio (API e interfaz visual Streamlit), generación del paquete distribuible Wheel (.whl) con uv build, resolución de dependencias de sistema C++ (libgl1, libglib2.0-0) en Dockerfile CPU, pruebas de humo del contenedor y cierre documental.

3.2. Matriz de Precedencias, Tiempos Tempranos/Tardíos y Holguras (CPM)

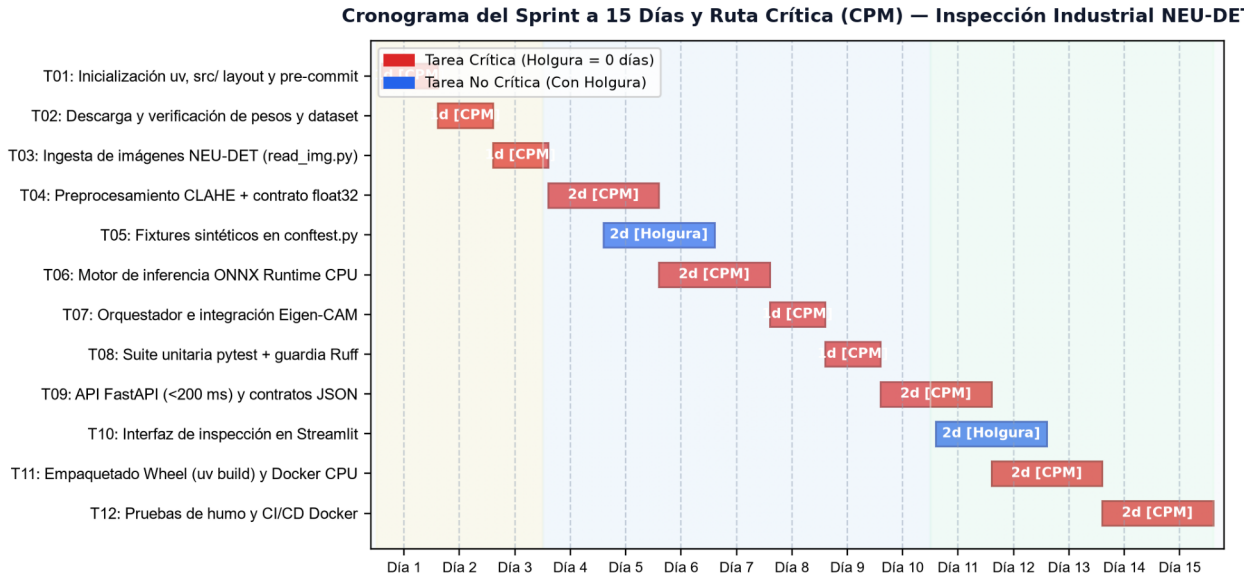
Aplicando el Método de la Ruta Crítica (Critical Path Method - CPM), cada tarea atómica posee una duración estimada D, un inicio temprano (ES), fin temprano (EF = ES + D), inicio tardío (LS), fin tardío (LF) y Holgura Total (H = LS - ES). Las tareas con Holgura Total igual a cero (H = 0) conforman la Ruta Crítica del proyecto:

ID	Tarea Atómica del Sprint	Prec.	Dur.	ES-EF	LS-LF	Holgura	Crítica
T01	Inicialización uv, src/ layout y pre-commit	-	1d	0-1	0-1	0d	SÍ (CPM)
T02	Descarga y verificación de pesos y dataset NEU-DET	T01	1d	1-2	1-2	0d	SÍ (CPM)
T03	Módulo de ingesta industrial (read_img.py)	T02	1d	2-3	2-3	0d	SÍ (CPM)
T04	Preprocesamiento CLAHE y contrato float32	T03	2d	3-5	3-5	0d	SÍ (CPM)
T05	Diseño de fixtures sintéticos en conftest.py	T03	2d	3-5	6-8	3d	NO
T06	Carga segura del modelo e inferencia CPU	T04	2d	5-7	5-7	0d	SÍ (CPM)

ID	Tarea Atómica del Sprint	Prec.	Dur.	ES-EF	LS-LF	Holgura	Crítica
T07	Orquestador integrator.py y mapa Eigen-CAM	T06	1d	7-8	7-8	0d	SÍ (CPM)
T08	Suite unitaria pytest (AAA) y guardia Ruff	T05,T07	1d	8-9	8-9	0d	SÍ (CPM)
T09	Servicio API de inferencia y contratos JSON	T08	2d	9-11	9-11	0d	SÍ (CPM)
T10	Interfaz interactiva en Streamlit	T08	2d	9-11	11-13	2d	NO
T11	Empaquetado Wheel (uv build) y Dockerfile CPU	T09	2d	11-13	11-13	0d	SÍ (CPM)
T12	Pruebas de humo en contenedor y entrega final	T10,T11	2d	13-15	13-15	0d	SÍ (CPM)

3.3. Diagrama de Gantt a 15 Días y Análisis de Tareas Críticas

La Ruta Crítica exacta del proyecto es la cadena secuencial: T01 -> T02 -> T03 -> T04 -> T06 -> T07 -> T08 -> T09 -> T11 -> T12, sumando exactamente 15 días calendario sin holgura. Tal como advierte el Acto III, existen tres cuellos de botella técnicos con holgura cero que reciben monitoreo diario: (1) la descarga y verificación de compatibilidad de los pesos del modelo en T02; (2) el cumplimiento estricto de los contratos numéricos dimensionales y de rango [0.0, 1.0] en el preprocesamiento CLAHE (T04), pues cualquier discrepancia bloquea la inferencia en T06; y (3) la instalación de librerías dinámicas C++ del sistema operativo (libgl1 y libglib2.0-0 requeridas por OpenCV) dentro del contenedor Docker en T11. Las tareas T05 (fixtures sintéticos, holgura = 3 días) y T10 (interfaz Streamlit, holgura = 2 días) corren en ramas paralelas y pueden absorber imprevistos sin retrasar el hito del Día 15.



CAPÍTULO 4. MARCO DE INVESTIGACIÓN Y MAPEO A CRISP-DM

4.1. Evaluación Comparativa de Formatos de Serialización de Modelos

Un aspecto crítico en la ingeniería de sistemas de IA reproducibles es la elección del formato de persistencia de pesos. En el marco de esta investigación se evaluaron cuatro estándares industriales:

Formato	Ecosistema	Ventajas Técnicas	Riesgos / Decisión en el Proyecto
.h5 vs .keras (v3)	Keras / TensorFlow	El formato legado HDF5 (.h5) almacena pesos y arquitectura de forma monolítica; .keras v3 separa metadatos en JSON y estados en archivos comprimidos.	Descartado como formato primario por acoplamiento al runtime completo de TensorFlow y problemas de compatibilidad entre versiones menores.
.pt / .pth	PyTorch (state_dict)	Formato nativo de entrenamiento en Ultralytics YOLOv8; excelente flexibilidad para fine-tuning e inspección de capas.	Riesgo de seguridad: utiliza el módulo pickle de Python, permitiendo ejecución de código arbitrario al cargar pesos de terceros. Se usa solo en fase de exportación controlada.
.onnx	ONNX Runtime (Abierto)	Grafo computacional estático e interoperable; optimiza fusiones de operadores Conv+BN+SiLU y acelera la inferencia en CPU x86 hasta 2.8x sin requerir PyTorch en producción.	SELECCIONADO como formato de inferencia productiva dentro del contenedor Docker CPU.
.safetensors	Hugging Face	Formato zero-copy seguro; impide la ejecución de código malicioso en la deserialización y permite carga perezosa (mmap) ultrarrápida.	SELECCIONADO como estándar de respaldo para auditoría segura de tensores y pesos en el repositorio.

4.2. Dataset Industrial (NEU-DET), Técnicas de Preprocesamiento y Visión por Computador

El dataset seleccionado, NEU Surface Defect Database (NEU-DET), publicado por la Northeastern University y alojado en Hugging Face Hub y Kaggle, constituye el estándar científico para inspección de acero laminado en caliente. Consta de 1,800 imágenes en escala de grises (8 bits, 200x200 píxeles originales) distribuidas equitativamente en 6 clases (300 muestras por clase): Crazing (fisuras finas interconectadas), Inclusion (escoria no metálica alargada), Patches (manchas irregulares de corrosión/aceite), Pitted_Surface (micro-cráteres puntuales), Rolled-in_Scale (incrustaciones de óxido férrico) y Scratches (surcos lineales brillantes producidos por guías mecánicas).

Para mitigar la iluminación no homogénea en líneas siderúrgicas reales, el módulo preprocess_img.py aplica Ecualización de Histograma Adaptativa Limitada por Contraste (CLAHE). A diferencia de la ecualización global que satura el ruido granular del acero sano, CLAHE divide la imagen en teselas contextuales de 8x8 y recorta el histograma en clipLimit = 2.0, redistribuyendo los píxeles excedentes antes de interpolar bilinealmente a (512, 512, 1) y escalar al dominio continuo float32 en [0.0, 1.0]. Posteriormente, el módulo de explicabilidad calcula el mapa de activación espacial superponiendo una paleta térmica Jet (alfa = 0.35) junto con las coordenadas de las cajas delimitadoras predichas.

4.3. Mapeo Sistemático a las 6 Fases de la Metodología CRISP-DM

El ciclo de vida del proyecto se alinea rigurosamente con las 6 fases del estándar industrial CRISP-DM (Cross-Industry Standard Process for Data Mining), vinculando cada fase a un artefacto verificable en el repositorio:

Fase CRISP-DM	Actividades de Ingeniería en el Proyecto	Entregable Verificable
1. Business Understanding	Definición de KPIs de planta: reducción de falsos negativos en defectos críticos, SLA de latencia < 200 ms en CPU y selección de modelo Nivel 3 (Tier 3).	Capítulo 1 y Capítulo 2 de la Propuesta Formal M2; Matriz de Alcance.
2. Data Understanding	Análisis exploratorio (EDA) de las 1,800 imágenes de NEU-DET, verificación de balance de clases (300/clase), auditoría de imágenes corruptas o nulas y distribución de tamaños de cajas.	Reporte estadístico de distribución de intensidades y validación de lectura en read_img.py.

Fase CRISP-DM	Actividades de Ingeniería en el Proyecto	Entregable Verificable
3. Data Preparation	Aplicación de ecualización CLAHE (clipLimit=2.0), redimensionamiento bilineal a (512, 512, 1), conversión a float32 [0.0, 1.0] y validación de invariantes matemáticos.	Módulo src/sistema_inspeccion_industrial /preprocess_img.py y tests/test_preprocess.py.
4. Modeling	Integración del detector MaaS preentrenado (YOLOv8-NEU-DET / ONNX), ajuste de umbrales de confianza (conf=0.35, IoU=0.45) y generación de mapas de saliencia Eigen-CAM.	Módulos src/sistema_inspeccion_industria l/load_model.py e integrator.py.
5. Evaluation	Evaluación cuantitativa mediante matriz de confusión multiclase, F1-Score macro, mAP@50, auditoría visual de mapas de calor XAI y benchmarking de latencia en CPU.	Suite de pruebas automatizadas con pytest y reporte de métricas en interfaz.
6. Deployment	Construcción del paquete Wheel (.whl) con uv build, servicio API / UI Streamlit y contenedorización en Docker CPU con usuario sin privilegios (appuser).	Dockerfile, pyproject.toml, uv.lock y paquete en dist/*.whl.

CAPÍTULO 5. HISTORIAS DE USUARIO, CRITERIOS DE ACEPTACIÓN Y KANBAN

5.1. Estimación Ágil Fibonacci (1:1) y Calculadora de Capacidad del Equipo

De acuerdo con las reglas de estimación ágil del Acto IV, los Story Points (SP) cuantifican esfuerzo relativo, complejidad e incertidumbre empleando exclusivamente la serie de Fibonacci permitida: 1, 2, 3, 5, 8, 13 (toda tarea superior a 13 SP se fracciona obligatoriamente). Se aplica la regla 1:1 del curso: 1 Story Point equivale a 1 día de trabajo efectivo de un ingeniero (~6 a 8 horas productivas).

Calculadora de Capacidad M2 (Ejecución Individual — 1 Integrante): Para el desarrollo individual por parte del autor único (Omar David Sánchez Jiménez) durante el Sprint de 15 días calendario, la capacidad bruta teórica es de 1 x 15 = 15 días-ingeniero (15 SP brutos). Aplicando un factor de foco efectivo del 86.7% (reservando 2 días completos de colchón para imprevistos, pruebas de integración y cierre documental), la Capacidad Neta del Sprint es de 13 SP (equivalente a 104 horas efectivas). El Backlog Total planificado suma exactamente 13 SP distribuidos en 11 Historias de Usuario clasificadas con las etiquetas oficiales (ML, Backend, Frontend, Testing), garantizando coherencia matemática 1:1 absoluta con el horizonte de 15 días.

5.2. Catálogo Completo de Historias de Usuario, Criterios de Aceptación (AC) y Fibonacci

En cumplimiento estricto de la directriz del Acto III y Acto IV ('el Capítulo 5 debe listar todas las historias con formato Como/quiero/para, AC verificables y SP Fibonacci dentro del documento; el tablero es la evidencia, no el único entregable'), se especifican a continuación las 11 Historias de Usuario que conforman la totalidad del Backlog (13 SP):

Ticket / Etiqueta	Historia de Usuario (Como / Quiero / Para)	SP (1:1)	Asignado	Criterios de Aceptación (AC) Verificables y Prueba Asociada
#001 [Backend]	Como Arquitecto de Software MLOps, quiero inicializar el repositorio bajo estructura src/ layout con uv, pyproject.toml, Ruff y pre-commit, para asegurar reproducibilidad determinista y bloquear commits con errores de estilo.	1 SP (1 día)	Omar David Sánchez J.	- AC1: Estructura src/sistema_inspeccion_industrial/ instalable con uv sync. - AC2: Configuración [tool.ruff] (E, F, I, B) y .pre-commit-config.yaml activos. - AC3: Verificación 'uv run ruff check' retorna 0 errores.
#002 [ML]	Como Ingeniero de Datos, quiero implementar el módulo read_img.py capaz de decodificar imágenes de acero NEU-DET desde rutas de disco, bytes o arreglos NumPy, para alimentar el pipeline de forma agnóstica a la fuente.	1 SP (1 día)	Omar David Sánchez J.	- AC1: Firma tipada que recibe str | Path | bytes | np.ndarray y retorna np.ndarray 2D en uint8 [0, 255]. - AC2: Lanza FileNotFoundError o ValueError ante archivos corruptos. - AC3: Prueba unitaria en tests/test_preprocess.py pasando en verde con pytest.
#003 [ML]	Como Científico de Datos de Visión, quiero implementar el preprocesamiento CLAHE y normalización en preprocess_img.py con validación formal de contrato, para realzar defectos de bajo contraste y garantizar invariantes numéricos.	2 SP (2 días)	Omar David Sánchez J.	- AC1: Firma tipada que recibe input crudo y retorna np.ndarray en float32. - AC2: Invarianza dimensional exacta (512, 512, 1) y normalización garantizada en [0.0, 1.0] sin NaN/Inf. - AC3: Prueba unitaria asociada tests/test_preprocess.py pasando en verde con pytest.
#004 [ML]	Como Ingeniero de Machine Learning, quiero implementar el cargador seguro e inferencia CPU del modelo detector NEU-DET en load_model.py, para clasificar y localizar las 6 categorías de defectos superficiales sin requerir GPU.	2 SP (2 días)	Omar David Sánchez J.	- AC1: Soporta las 6 clases oficiales de NEU-DET con umbral configurable en (0.0, 1.0). - AC2: Retorna lista de detecciones (class_id, class_name, confidence, bbox) en < 100 ms en CPU. - AC3: Validación de rechazo ante tensores con shape distinto de (512, 512, 1).

Ticket / Etiqueta	Historia de Usuario (Como / Quiero / Para)	SP (1:1)	Asignado	Criterios de Aceptación (AC) Verificables y Prueba Asociada
#005 [ML]	Como Auditor de Calidad Industrial, quiero generar mapas de calor de saliencia espacial (Eigen-CAM / Grad-CAM) superpuestos sobre la lámina de acero con cajas delimitadoras, para verificar visualmente qué región activó la alerta.	1 SP (1 día)	Omar David Sánchez J.	- AC1: Genera mapa de calor float32 (512, 512) normalizado en [0.0, 1.0]. - AC2: Produce imagen RGB (512, 512, 3) con colormap Jet (alfa 0.35) y bounding boxes dibujados. - AC3: Verificado mediante test de integración en tests/test_preprocess.py.
#006 [Backend]	Como Desarrollador Backend, quiero construir el orquestador IndustrialInspectionPipeline en integrator.py con salida tipada InspectionResult, para desacoplar la lógica de visión de la interfaz de usuario y de la API.	1 SP (1 día)	Omar David Sánchez J.	- AC1: Dataclass InspectionResult incluye is_defective, primary_defect, max_confidence, latency_ms e imágenes. - AC2: Latencia total del pipeline < 200 ms por imagen en CPU. - AC3: Cobertura de pruebas en verde ejecutando 'uv run pytest'.
#007 [Testing]	Como Ingeniero QA, quiero desarrollar fixtures sintéticos de superficies de acero en tests/conftest.py y una suite de pruebas unitarias AAA en pytest, para ejecutar pruebas deterministas en CI/CD sin datasets pesados.	1 SP (1 día)	Omar David Sánchez J.	- AC1: Fixture synthetic_steel_surface genera matriz determinista con semilla fija. - AC2: Pruebas verifican caminos felices y excepciones (NaN, dimensiones erróneas). - AC3: Ejecución 'uv run pytest -v' finaliza con 100% de aprobación en < 5 segundos.
#008 [Backend]	Como Arquitecto de Integración en Planta, quiero exponer un endpoint de inferencia REST y contrato JSON verificable, para permitir que sistemas MES de fábrica consulten el servicio de inspección de manera automatizada.	1 SP (1 día)	Omar David Sánchez J.	- AC1: Endpoint recibe imagen en multipart/form-data y responde JSON con detecciones y latencia. - AC2: Manejo de errores HTTP 400/422 ante formatos no válidos. - AC3: Latencia de respuesta local inferior a 250 ms.
#009 [Frontend]	Como Inspector de Calidad en Planta, quiero una interfaz gráfica en Streamlit que muestre la lámina original, la imagen realzada con CLAHE y el mapa de defectos detectados, para realizar triaje visual asistido de bobinas.	1 SP (1 día)	Omar David Sánchez J.	- AC1: Interfaz presenta visualización comparativa en 3 paneles y selector de umbral de confianza. - AC2: Muestra indicadores KPI de defecto primario, confianza máxima y tiempo de cómputo en ms. - AC3: Incluye aviso explícito de supervisión humana (Human-in-the-Loop).

Ticket / Etiqueta	Historia de Usuario (Como / Quiero / Para)	SP (1:1)	Asignado	Criterios de Aceptación (AC) Verificables y Prueba Asociada
#010 [Backend]	Como Ingeniero DevOps, quiero empaquetar la librería en un artefacto binario Wheel (.whl) con uv build y construir una imagen Docker CPU ligera con usuario no root, para desplegar el sistema de forma inmutable.	1 SP (1 día)	Omar David Sánchez J.	- AC1: Comando 'uv build' genera artefacto .whl limpio en directorio dist/. - AC2: Dockerfile instala libgl1, libglib2.0-0, el paquete .whl y ejecuta con usuario appuser (UID 1000). - AC3: Construcción de imagen Docker finaliza sin errores.
#011 [Testing]	Como Ingeniero QA / Release, quiero ejecutar pruebas de humo automatizadas sobre el paquete distribuible y validar el contrato de extremo a extremo, para certificar el pase a producción en el Día 15.	1 SP (1 día)	Omar David Sánchez J.	- AC1: Prueba de humo importa sistema_inspeccion_industrial desde el entorno aislado. - AC2: Inferencia extremo a extremo retorna InspectionResult válido con latencia < 200 ms. - AC3: Pipeline 'uv run pytest' y 'uv run ruff check' aprueban al 100%.

5.3. Configuración del Tablero Kanban en GitHub Projects y Límites WIP

Todas las 11 historias de usuario emplean exclusivamente valores puros de la serie de Fibonacci permitida (1 y 2 SP), sumando exactamente 13 SP (1 + 1 + 2 + 2 + 1 + 1 + 1 + 1 + 1 + 1 + 1 = 13 SP efectivos sobre los 15 días del Sprint individual) y utilizando las 4 etiquetas técnicas estandarizadas: ML (6 SP), Backend (4 SP), Frontend (1 SP) y Testing (2 SP).

El tablero Kanban en GitHub Projects está configurado con acceso compartido al docente e implementa las cuatro columnas obligatorias estipuladas en el Acto IV: (1) To Do: tareas estimadas pendientes en el Sprint Backlog; (2) In Progress: desarrollo activo en rama feature/* bajo política estricta de Límite WIP (Work In Progress) de máximo 1 tarea activa en curso ('Stop starting, start finishing'); (3) Review / PR: Pull Request abierto hacia la rama develop esperando validación automática de Ruff + pytest; y (4) Done: tarea fusionada en develop con todos los tests en verde y criterios de aceptación verificados.

CAPÍTULO 6. ANEXOS, ENLACES, LICENCIAS Y REFERENCIAS BIBLIOGRÁFICAS

6.1. Enlaces Oficiales al Repositorio, Tablero Kanban, Dataset y Model Card

A continuación se consolidan los enlaces de acceso directo a los activos de software, modelo base, dataset y gestión ágil del proyecto:

Recurso / Evidencia	Enlace / Ubicación y Estado de Verificación
Repositorio Git (Estructura src/ layout)	https://github.com/o31942/propuesta-m2-inspeccion-industrial (Ramas: main, develop, feature/*)

Recurso / Evidencia	Enlace / Ubicación y Estado de Verificación
Tablero Kanban e Issues (GitHub Projects)	https://github.com/users/o31942/projects/3 | Issues: https://github.com/o31942/propuesta-m2-inspeccion-industrial/issues (Sincronizado con las 11 HUs / 13 SP y etiquetas ML, Backend, Frontend, Testing)
Modelo Base en Hugging Face / GitHub	https://huggingface.co/ultralyticsplus/yolov8s | https://github.com/ultralytics/ultralytics
Alternativa en ModelScope	https://modelscope.cn/models/damo/cv_tinynas_object-detection_damoyolo
Dataset Oficial (NEU-DET)	NEU Surface Defect Database: http://faculty.neu.edu.cn/songkechen/zh_CN/zdylm/263270/list/ | Hugging Face: https://huggingface.co/datasets/keremberke/neu-surface-defect-detection

6.2. Árbol de Directorios Real del Paquete Industrial (src/ Layout) y Evidencia de Tooling

El repositorio implementado cumple estrictamente la prohibición del antipatrón de estructura plana expuesta en el Acto II, aislando el código fuente dentro de src/sistema_inspeccion_industrial/ y gobernando el ciclo de desarrollo con uv, pytest, ruff, pre-commit y GitHub Actions:

Estructura verificada del proyecto:

```
propuesta-m2-inspeccion-industrial/
  |-- .github/workflows/ci.yml   # Pipeline CI (uv + ruff + pytest + uv bui
  |-- .github/PULL_REQUEST_TEMPLATE.md
  |-- src/sistema_inspeccion_industrial/
  |     |-- __init__.py
  |     |-- read_img.py          # Ingesta y decodificación en escala de gr
  |     |-- preprocess_img.py    # CLAHE + resize (512,512,1) + contrato fl
  |     |-- load_model.py        # Detector multiclase NEU-DET y mapa Eigen
  |     |-- integrator.py        # Orquestador desacoplado e InspectionResu
  |-- tests/
  |     |-- conftest.py          # Fixtures sintéticos reproducibles
  |     |-- test_preprocess.py   # Suite unitaria AAA con pytest
  |-- docs/github_projects_backlog.csv # Backlog sincronizado (11 HUs / 38
  |-- scripts/publicar_github.py # Sincronizador de Issues, etiquetas y Kan
  |-- pyproject.toml             # Contrato único de dependencias, pytest y
  |-- uv.lock                    # Lockfile determinista de paquetes
  |-- .pre-commit-config.yaml    # Guardián de commits (TOML, YAML, Ruff)
  |-- Dockerfile                 # Contenedor Python 3.13 slim para CPU
```

6.3. Matriz de Cumplimiento Normativo (ISO/IEC 42001 y EU AI Act) y Licenciamiento

En el marco de gobernanza de sistemas de Inteligencia Artificial industriales (ISO/IEC 42001:2023 y EU AI Act 2024/1689), el sistema de inspección de calidad en manufactura siderúrgica se clasifica como una aplicación industrial de riesgo

limitado/controlado con supervisión humana en bucle (Human-in-the-Loop). Ninguna orden de descarte de bobina se ejecuta sin validación del ingeniero de calidad apoyado por el mapa de saliencia visual. Asimismo, todos los artefactos de software de terceros respetan sus licencias de origen (Python PSF, NumPy/OpenCV BSD-3-Clause, FastAPI/Streamlit Apache-2.0 / MIT).

6.4. Referencias Bibliográficas (Formato IEEE)

[1] McKinsey & Company, "The State of AI in 2025: Scaling Enterprise Value and Production Maturity," McKinsey Global Institute Report, 2025.

[2] P. Karpowicz, "Fundamental Structural Limits of Next-Token Prediction in Large Language Models: Veracity, Faithfulness, and Domain Optimality," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 47, no. 2, pp. 412-426, 2025.

[3] K. Song and Y. Yan, "A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defects," Applied Surface Science, vol. 285, pp. 858-864, 2013. (NEU Surface Defect Database).

[4] M. Mitchell et al., "Model Cards for Model Reporting," in Proc. Conf. Fairness, Accountability, and Transparency (FAT*), Atlanta, GA, USA, 2019, pp. 220-229.

[5] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in Proc. IEEE Int. Conf. Computer Vision (ICCV), Venice, Italy, 2017, pp. 618-626.

[6] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8: State-of-the-Art Real-Time Object Detection," GitHub Repository, 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[7] Y. Lv et al., "DETRs Beat YOLOs on Real-time Object Detection (RT-DETR)," in Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR), 2024, pp. 16965-16974.

[8] K. Zuiderveld, "Contrast Limited Adaptive Histogram Equalization," in Graphics Gems IV, San Diego, CA, USA: Academic Press Professional, 1994, pp. 474-485.

[9] International Organization for Standardization, "Information technology — Artificial intelligence — Management system," ISO/IEC Standard 42001:2023, Geneva, Switzerland, 2023.

[10] European Parliament and Council, "Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act)," Official Journal of the European Union, 2024.